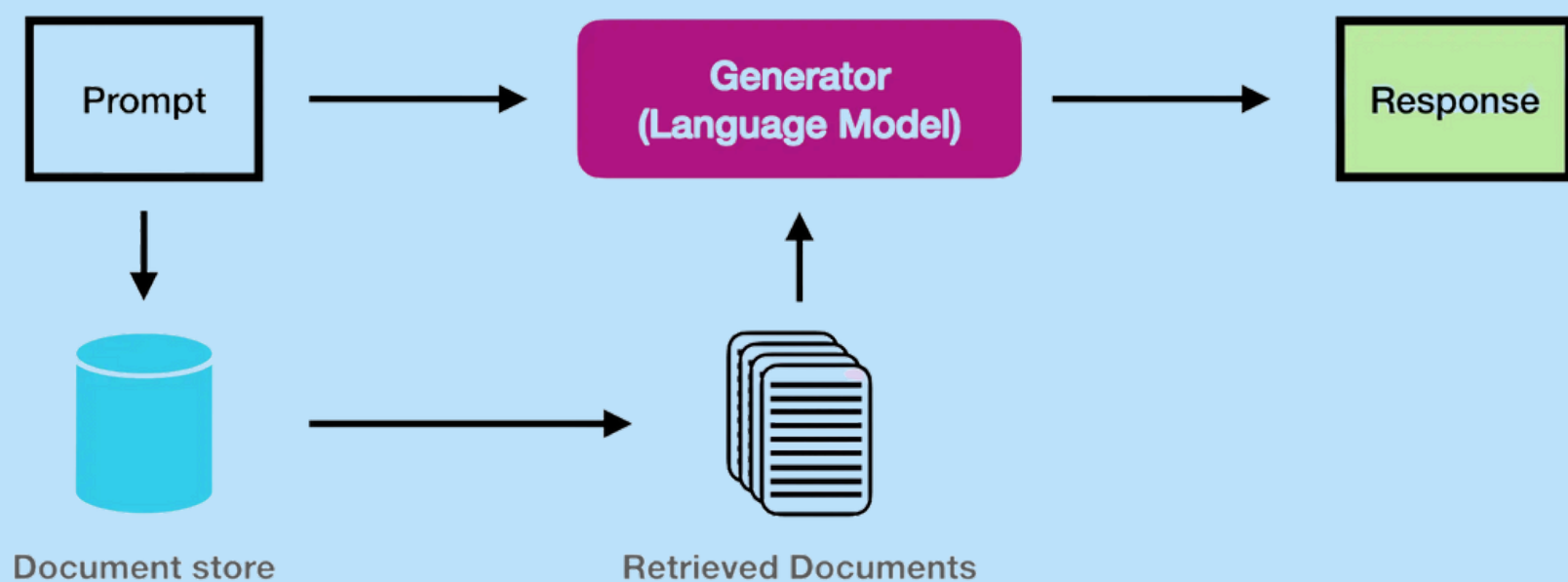


A Hands-On

RAG WITH LANGGRAPH



What is RAG?

RAG (Retrieval-Augmented Generation) combines information retrieval with LLMs to answer questions using your own documents. Instead of relying on training data, RAG retrieves relevant context from your documents and uses it to generate accurate answers.

WHY RAG?

- Answer questions from your own documents
- No need to retrain models
- Reduces hallucinations by grounding in context
- Works with any LLM and document type



Install Required Libraries

```
pip install -U langgraph langchain langchain-  
community langchain-text-splitters chromadb  
tiktoken  
pip install -U langchain-openai
```

Add your documents in directory

```
docs/  
faq.txt  
handbook.txt  
product_notes.md
```



Load documents

- Load all files from /docs
- Convert into Document objects

CODE

```
# index.py
from pathlib import Path
from langchain_community.document_loaders import TextLoader

DOCS_DIR = Path("docs")

def load_docs():
    docs = []
    for fp in DOCS_DIR.glob("*"):
        if fp.is_file():
            docs.extend(TextLoader(str(fp),
encoding="utf-8").load())
    return docs
```



Split into chunks

- Break large documents into smaller, searchable pieces
- 800 chars per chunk with 120 char overlap
- Overlap ensures context isn't lost at boundaries

CODE

```
# index.py  
from langchain_text_splitters import  
RecursiveCharacterTextSplitter  
  
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=800,  
    chunk_overlap=120  
)  
chunks = splitter.split_documents(load_docs())
```



Create embeddings + store in Chroma

- Convert text chunks into numerical vectors (embeddings)
- Store in Chroma for lightning-fast similarity search
- Run once: `python index.py` (requires `OPENAI_API_KEY`)

CODE

```
# index.py  
from langchain_openai import OpenAIEmbeddings  
from langchain_community.vectorstores import  
Chroma  
  
embeddings = OpenAIEmbeddings()  
  
Chroma.from_documents(  
    documents=chunks,  
    embedding=embeddings,  
    persist_directory="chroma_db",  
    collection_name="rag_docs",  
)  
print("✅ Index built")
```



Create LangGraph state

- State = Data that flows through your graph
- Contains: question (input), docs (retrieved), answer (output)
- TypedDict ensures type safety

CODE

```
# rag_graph.py  
from typing import TypedDict, List  
from langchain_core.documents import Document  
  
class RAGState(TypedDict):  
    question: str  
    docs: List[Document]  
    answer: str
```



Build Retrieve Node

- First node: Find most relevant document chunks
- Searches vector DB for top 4 matching chunks
- Returns chunks in state for next node

CODE

```
# rag_graph.py
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

vectorstore = Chroma(
    persist_directory="chroma_db",
    collection_name="rag_docs",
    embedding_function=OpenAIEmbeddings(),
)
retriever =
vectorstore.as_retriever(search_kwargs={"k": 4})

def retrieve(state: RAGState) → RAGState:
    q = state["question"]
    docs = retriever.invoke(q)
    return {"question": q, "docs": docs, "answer":
    ""}
```



Set Up LLM & Prepare Context

- Initialize ChatOpenAI with gpt-4o-mini
- Extract question and docs from state
- Combine document chunks into context string

CODE

```
# rag_graph.py
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage,
HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini",
temperature=0)

def generate(state: RAGState) → RAGState:
    q = state["question"]
    docs = state["docs"]

    # Combine all document chunks into context
    context = "\n\n".join([d.page_content for d in
docs])
```



Build Generate Function

- Create prompt with system message (answer only from context)
- Inject context and question into HumanMessage
- Call LLM and return answer in state

CODE

```
# rag_graph.py (continued)
messages = [
    SystemMessage(content=(
        "Answer using ONLY the context. "
        "If missing, say you don't have that"
        "info in the documents."
    )),
    HumanMessage(content=f"Question:
{q}\n\nContext:\n{context}")
]

resp = llm.invoke(messages)
return {**state, "answer": resp.content}
```



Connect Nodes into LangGraph

- Define the flow: `START → retrieve → generate → END`
- LangGraph handles execution automatically
- Easy to add more nodes later (filtering, reranking, etc.)

CODE

```
# rag_graph.py
from langgraph.graph import StateGraph, START, END

def build_graph():
    g = StateGraph(RAGState)
    g.add_node("retrieve", retrieve)
    g.add_node("generate", generate)

    g.add_edge(START, "retrieve")
    g.add_edge("retrieve", "generate")
    g.add_edge("generate", END)

    return g.compile()

rag_app = build_graph()
```



Run Your RAG App

- Simple CLI loop: Ask questions, get answers
- See which document chunks were used
- Your RAG chatbot is ready! 🎉

CODE

```
# run.py
from rag_graph import rag_app

while True:
    q = input("\nAsk: ").strip()
    if not q:
        break

    result = rag_app.invoke({"question": q,
"docs": [], "answer": ""})
    print("\nAnswer:\n", result["answer"])
    print("\nChunks used:", len(result["docs"]))
```



Want more content like this?



Tap that follow button and
stay in the loop!



Like



Comment



Share



Save